

ISYE 6501 HW 1

Joseph Shin

2025-08-19

```
library(kernlab)
```

```
## Warning: package 'kernlab' was built under R version 4.3.3
```

```
data = read.csv("credit_card_data-headers.txt", header = TRUE, sep = "\t")
```

2.2.1: Using the vanilladot kernel

C = 100

```
model = ksvm(as.matrix(data[,1:10]), as.factor(data[,11]), type = "C-svc", kernel = "vanilladot", C = 100)
```

```
## Setting default kernel parameters
```

```
model
```

```
## Support Vector Machine object of class "ksvm"
```

```
##
```

```
## SV type: C-svc (classification)
```

```
## parameter : cost C = 100
```

```
##
```

```
## Linear (vanilla) kernel function.
```

```
##
```

```
## Number of Support Vectors : 189
```

```
##
```

```
## Objective Function Value : -17887.92
```

```
## Training error : 0.136086
```

```
a = colSums(model@xmatrix[[1]] * model@coef[[1]])
```

```
a
```

```
##           A1           A2           A3           A8           A9
```

```
## -0.0010065348 -0.0011729048 -0.0016261967  0.0030064203  1.0049405641
```

```
##           A10          A11          A12          A14          A15
```

```
## -0.0028259432  0.0002600295 -0.0005349551 -0.0012283758  0.1063633995
```

```
a0 = -model@b
```

```
a0
```

```
## [1] 0.08158492
```

```
pred = predict(model, data[,1:10])
pred
```

[illegible]

```
sum(pred == data[,11]) / nrow(data)
```

```
## [1] 0.8639144
```

```
y = factor(data[,11])
false_positives <- sum(pred == 1 & y == 0)

x = as.matrix(data[,1:10])
y = factor(data[,11], levels = c(0,1))
cm = table(actual = y, pred = pred)
FP <- cm["0", "1"] # false positives: predicted 1, actual 0
FN <- cm["1", "0"] # false negatives: predicted 0, actual 1
FP; FN
```

```
## [1] 72
```

```
## [1] 17
```

C = 1000000

```
model2 = ksvm(as.matrix(data[,1:10]), data[,11], type = "C-svc", kernel = "vanilladot", C = 1000000, sc
```

```
## Setting default kernel parameters
```

model2

```
## Support Vector Machine object of class "ksvm"
##
## SV type: C-svc (classification)
## parameter : cost C = 1e+06
##
## Linear (vanilla) kernel function.
##
## Number of Support Vectors : 427
##
## Objective Function Value : -46182140
## Training error : 0.374618
```

```
a = colSums(model2@xmatrix[[1]] * model2@coef[[1]])
a0 = -model2@b
a0
```

```
## [1] -0.1281168
```

```
pred = predict(model2, data[,1:10])
pred
```

```
## [1] 0 1 1 1 1 0 1 1 0 1 1 0 1 0 1 0 0 1 1 1 0 0 1 1 1 1 1 0 0 0 0 1 1 1 0 0
## [38] 1 0 1 1 1 0 1 1 1 1 0 0 0 1 1 1 1 1 1 1 1 0 0 0 0 1 1 1 1 1 1 0 0 0 0 0 0
## [75] 0 0 1 0 0 0 1 0 1 1 0 1 1 0 0 1 1 1 0 1 1 1 0 0 0 1 0 0 1 1 1 0 1 0 1 1 1
## [112] 0 1 0 1 0 1 1 1 1 1 0 1 0 1 1 0 1 0 0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1 0 1 1
## [149] 1 1 1 1 1 1 0 1 1 0 0 1 1 1 1 1 0 1 1 1 1 0 1 1 1 1 0 1 1 1 1 1 0 0 1 1 1
## [186] 1 0 0 0 1 1 0 1 0 0 1 1 1 0 1 0 0 1 0 1 1 0 0 0 0 1 0 1 1 1 1 1 1 1 1 1 1
## [223] 0 1 1 0 0 1 1 1 1 1 0 1 0 1 0 1 1 1 0 1 0 1 0 0 0 0 0 1 0 1 1 1 0 0 0 0 0
## [260] 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 1 0 0 1 0 1 0 0 0
## [297] 1 1 1 1 1 0 1 0 1 0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0
## [334] 1 0 1 1 0 0 0 1 0 1 1 1 0 0 0 0 0 0 1 1 0 0 1 0 1 0 0 0 0 0 1 1 1 1 1 1 1
## [371] 0 0 0 0 0 1 0 1 0 1 1 0 0 0 1 1 0 1 0 0 0 0 0 1 0 0 0 0 1 0 1 0 0 0 0 0 0
## [408] 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 1 1 0 0 0 0 0 0
## [445] 1 0 1 0 0 0 0 0 1 0 1 0 1 0 1 0 1 0 0 0 0 0 0 0 0 0 1 1 1 1 0 0 1 0 0 1 1 1
## [482] 1 0 1 1 1 1 1 1 1 0 1 0 1 0 1 1 1 0 1 1 0 1 1 0 0 0 1 1 0 1 0 1 1 1 1 1 1
## [519] 1 0 0 1 0 0 1 0 1 0 0 1 1 1 0 1 0 1 1 1 0 1 0 1 1 1 0 1 1 0 1 0 0 1 0 0 0
## [556] 1 1 1 1 0 1 1 0 0 0 1 0 1 1 1 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0
## [593] 0 0 0 1 1 0 1 0 1 0 0 0 1 0 0 0 1 0 1 1 1 0 1 0 1 1 1 1 1 0 1 1 1 0 0 1 0
## [630] 0 0 1 1 0 0 0 0 1 0 0 1 0 1 1 0 0 1 0 0 0 1 0 0 1
```

```
sum(pred == data[,11]) / nrow(data)
```

```
## [1] 0.6253823
```

```
x = as.matrix(data[,1:10])
y = factor(data[,11], levels = c(0,1))
cm <- table(actual=y, pred=pred)
FP <- cm["0","1"] # false positives: predicted 1, actual 0
FN <- cm["1","0"] # false negatives: predicted 0, actual 1
FP; FN
```

```
## [1] 129
```



```
a0 = -model4@b
a0
```

```
## [1] -0.2226155
```

```
pred = predict(model4, data[,1:10])
pred
```

[illegible]

```
sum(pred == data[,11]) / nrow(data)
```

```
## [1] 0.8379205
```

```
x = as.matrix(data[,1:10])
y = factor(data[,11], levels = c(0,1))
cm <- table(actual=y, pred=pred)
FP <- cm["0", "1"] # false positives: predicted 1, actual 0
FN <- cm["1", "0"] # false negatives: predicted 0, actual 1
FP; FN
```

```
## [1] 28
```

```
## [1] 78
```

$$C = .0001$$

```
model5 = ksvm(as.matrix(data[,1:10]), data[,11], type = "C-svc", kernel = "vanilladot", C = .0001, scal
```

```
## Setting default kernel parameters
```

```
model5
```

```
## Support Vector Machine object of class "ksvm"
##
## SV type: C-svc (classification)
## parameter : cost C = 1e-04
##
## Linear (vanilla) kernel function.
##
## Number of Support Vectors : 592
##
## Objective Function Value : -0.0575
## Training error : 0.452599
```

```
a = colSums(model5@xmatrix[[1]] * model5@coef[[1]])
a
```

```
##           A1           A2           A3           A8           A9
## -0.0004319557  0.0065415940  0.0087400797  0.0179762408  0.0412707034
##           A10          A11          A12          A14          A15
## -0.0239617452  0.0233201856 -0.0008017346 -0.0044802708  0.0102239706
```

```
a0 = -model5@b
a0
```

```
## [1] -0.9023439
```

```
pred = predict(model5, data[,1:10])
pred
```

```
## [1] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [38] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [75] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [112] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [149] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [186] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [223] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [260] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [297] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [334] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [371] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [408] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [445] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [482] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [519] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [556] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [593] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [630] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
## [1] 0.5474006
```

Testing different kernels (I will be testing different C values to try to increase the accuracy percentage of each model as well as minimizing the number of false positives.)

```
rbfmod = ksvm(as.matrix(data[,1:10]), data[,11], type = "C-svc", kernel = "rbfdot", C = 1, scaled = TRUE)
rbfmod
```

```
##
## SV type: C-svc (classification)
## parameter : cost C = 1
##
## Gaussian Radial Basis kernel function.
## Hyperparameter : sigma = 0.0898233465613524
##
## Number of Support Vectors : 267
##
## Objective Function Value : -188.9062
## Training error : 0.129969
```

##	A1	A2	A3	A8	A9	A10
##	0.5611802	-1.7239286	3.7695941	12.8784241	30.3319535	-6.3535385
##	A11	A12	A14	A15		
##	18.0692431	-4.6845091	-15.7810692	26.4508627		

```
## [1] 0.4204407
```

[illegible]

```
## [297] 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
## [334] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [371] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [408] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [445] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [482] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [519] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [556] 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0
## [593] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [630] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
sum(pred == data[,11]) / nrow(data)
```

```
## [1] 0.8700306
```

```
x = as.matrix(data[,1:10])
y = factor(data[,11], levels = c(0,1))
cm <- table(actual=y, pred=pred)
FP <- cm["0","1"] # false positives: predicted 1, actual 0
FN <- cm["1","0"] # false negatives: predicted 0, actual 1
FP; FN
```

```
## [1] 68
```

```
## [1] 17
```

Using polydot kernel:

```
polymod = ksvm(as.matrix(data[,1:10]), data[,11], type = "C-svc", kernel = "polydot", C = .001, scaled =
```

```
## Setting default kernel parameters
```

```
polymod
```

```
## Support Vector Machine object of class "ksvm"
##
## SV type: C-svc (classification)
## parameter : cost C = 0.001
##
## Polynomial kernel function.
## Hyperparameters : degree = 1 scale = 1 offset = 1
##
## Number of Support Vectors : 556
##
## Objective Function Value : -0.438
## Training error : 0.16208
```

```
a = colSums(polymod@xmatrix[[1]] * polymod@coef[[1]])
a
```

```
##           A1           A2           A3           A8           A9           A10
## -0.002159778  0.032338170  0.046612449  0.111223162  0.375305335 -0.202026081
##           A11           A12           A14           A15
##  0.169560847 -0.004923501 -0.025210266  0.081189766
```

```
a0 = -polymod@b
a0
```

```
## [1] -0.2226155
```

```
pred = predict(polymod, data[,1:10])
pred
```

```
## [1] 1 1 0 1 0 0 1 0 0 0 0 0 0 0 1 1 1 1 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [38] 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 1 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 0 1 1 0
## [75] 1 0 0 1 0 1 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 1 1 1 1 1 1 0 0 1 1 1 1 0 1 1 1
## [112] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [149] 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 0 1 0 0 1 1 1 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1
## [186] 0 0 0 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 0 1 1 1 0 1 1 1 1 1 1 1 0 0 0 0 0 1
## [223] 0 1 1 1 1 1 1 1 0 1 0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0
## [260] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [297] 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [334] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [371] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [408] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [445] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [482] 1 1 1 1 0 0 0 0 0 1 1 1 1 1 1 1 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [519] 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 0 0 0 1 0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 0 0
## [556] 0 1 1 1 1 1 1 1 1 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [593] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [630] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
sum(pred == data[,11]) / nrow(data)
```

```
## [1] 0.8379205
```

```
x = as.matrix(data[,1:10])
y = factor(data[,11], levels = c(0,1))
cm <- table(actual=y, pred=pred)
FP <- cm["0","1"] # false positives: predicted 1, actual 0
FN <- cm["1","0"] # false negatives: predicted 0, actual 1
FP; FN
```

```
## [1] 28
```

```
## [1] 78
```

Using tanhdot kernel:

```
tanhmod = ksvm(as.matrix(data[,1:10]), data[,11], type = "C-svc", kernel = "tanhdot", C = 100, scaled =
```

```
## Setting default kernel parameters
```

```
tanhmod
```

```
## Support Vector Machine object of class "ksvm"
```

```
##
```

```
## SV type: C-svc (classification)
```

```
## parameter : cost C = 100
```

```
##
```

```
## Hyperbolic Tangent kernel function.
```

```
## Hyperparameters : scale = 1 offset = 1
```

```
##
```

```
## Number of Support Vectors : 184
```

```
##
```

```
## Objective Function Value : -18741416
```

```
## Training error : 0.278287
```

```
a = colSums(tanhmod@xmatrix[[1]] * tanhmod@coef[[1]])
```

```
a
```

```
##          A1          A2          A3          A8          A9
```

```
## 1.609693e+03 -6.068929e+02 -7.403735e+01 8.691337e+02 3.916100e+03
```

```
##          A10          A11          A12          A14          A15
```

```
## -1.733724e-12 2.096277e+03 -1.092974e+03 2.694266e+02 1.836964e+03
```

```
a0 = -tanhmod@b
```

```
a0
```

```
## [1] 80.08727
```

```
pred = predict(tanhmod, data[,1:10])
```

```
pred
```

```
## [1] 1 1 0 0 0 1 1 0 1 1 0 1 0 0 1 0 0 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 0 0
```

```
## [38] 1 1 1 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 0
```

```
## [75] 1 1 1 1 0 1 0 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 0 1 1 0 1 1 0 1 1 1 0 1
```

```
## [112] 1 1 1 1 1 1 1 1 1 1 0 0 1 1 0 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 0
```

```
## [149] 0 0 1 1 1 0 0 1 0 0 1 0 0 1 0 1 0 1 1 1 0 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1
```

```
## [186] 1 1 1 1 0 0 0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 1 0 0 0
```

```
## [223] 0 1 1 0 1 1 1 1 1 1 1 0 1 1 1 0 1 1 1 1 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1
```

```
## [260] 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 1 0 0 0 1 1 0 1 0 0 1 1 0 0 0 0 0 0 0 0
```

```
## [297] 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
```

```
## [334] 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
## [371] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0
```

```
## [408] 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 1 0 1 0
```

```
## [445] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 0 0 0 0 0 0 0 1 0 0 0 1 0 0 1 0 0 1 1 1 1
```

```
## [482] 1 1 0 0 0 0 0 1 1 0 1 0 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 0 0 1 1 1 1 1 1 0 0
```

```
## [519] 0 1 0 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 0 0 0 0 1 1 0 0 1 1 0 1 1 1 0 0 0 1
```

```
## [556] 1 0 1 1 1 0 0 1 0 0 1 1 1 1 0 1 0 0 1 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0
```

```
## [593] 0 0 0 0 0 1 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 0 0 0 0 1 0 0 0 0
```

```
## [630] 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0
```

```
sum(pred == data[,11]) / nrow(data)
```

```
## [1] 0.7217125
```

```
x = as.matrix(data[,1:10])
y = factor(data[,11], levels = c(0,1))
cm <- table(actual=y, pred=pred)
FP <- cm["0","1"] # false positives: predicted 1, actual 0
FN <- cm["1","0"] # false negatives: predicted 0, actual 1
FP; FN
```

```
## [1] 91
```

```
## [1] 91
```

Using anovadot kernel:

```
anovamod = ksvm(as.matrix(data[,1:10]), data[,11], type = "C-svc", kernel = "anovadot", C = .01, scaled
```

```
## Setting default kernel parameters
```

```
anovamod
```

```
## Support Vector Machine object of class "ksvm"
##
## SV type: C-svc (classification)
## parameter : cost C = 0.01
##
## Anova RBF kernel function.
## Hyperparameter : sigma = 1 degree = 1
##
## Number of Support Vectors : 371
##
## Objective Function Value : -2.7787
## Training error : 0.137615
```

```
a = colSums(anovamod@xmatrix[[1]] * anovamod@coef[[1]])
a
```

```
##           A1           A2           A3           A8           A9           A10
## -0.007954644 0.023641274 0.024724331 0.222923331 1.811439596 -0.093678784
##           A11           A12           A14           A15
## 0.296930152 0.007104125 -0.164190900 0.516275284
```

```
a0 = -anovamod@b
a0
```

```
## [1] 0.1497898
```

```
pred = predict(anovamod, data[,1:10])
pred
```

[illegible]

```
sum(pred == data[,11]) / nrow(data)
```

```
## [1] 0.8623853
```

```
x = as.matrix(data[,1:10])
y = factor(data[,11], levels = c(0,1))
cm <- table(actual=y, pred=pred)
FP <- cm["0","1"] # false positives: predicted 1, actual 0
FN <- cm["1","0"] # false negatives: predicted 0, actual 1
FP; FN
```

```
## [1] 72
```

```
## [1] 18
```

Question 2.2.3: k-nearest-neighbors classification

```
library(kknn)
```

```
## Warning: package 'knnn' was built under R version 4.3.3
```

```
accuracy_knn = function(x) {  
  pred = rep(0, (nrow(data)))  
  
  for (i in 1:nrow(data)) {  
  
    model = kknn(R1 ~. , train = data[-i,], test = data[i,], k = x, scale = TRUE, kernel = "op
```

```

pred[i] <- as.integer(fitted(model)+0.5)
}

accuracy = sum(pred == data[,11]) / nrow(data)
return(accuracy)
}

accuracy_vector <- 1:20
for (x in 1:20){
  accuracy_vector[x] = accuracy_knn(x) * 100
}
accuracy_vector

```

```

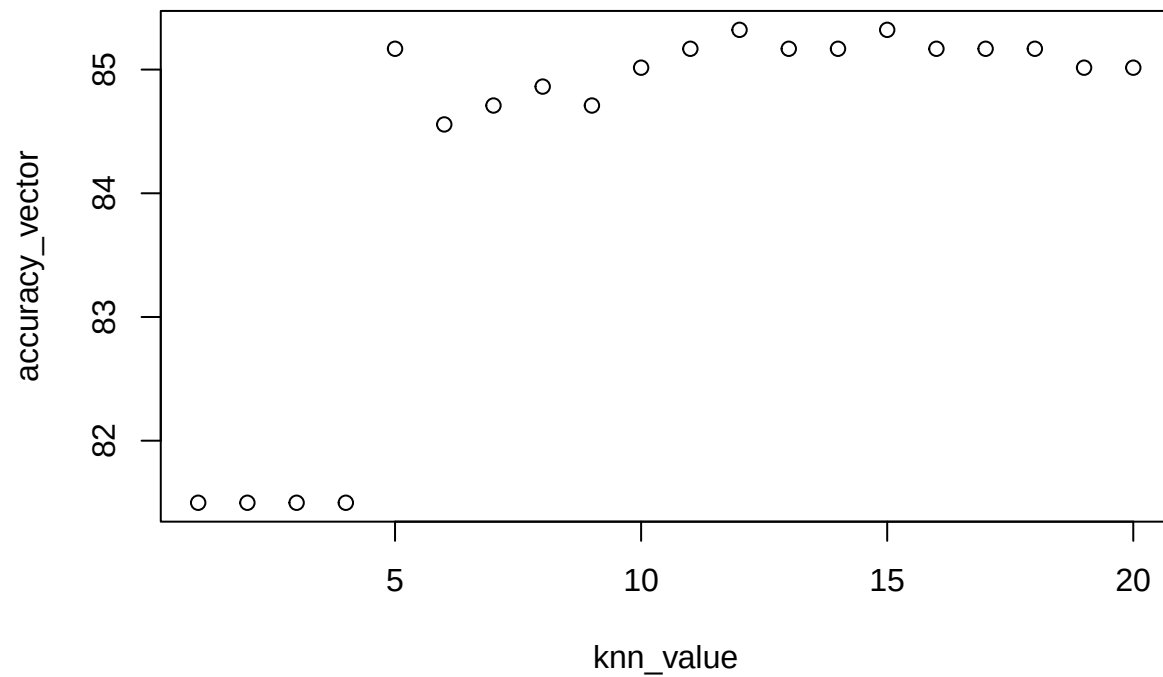
## [1] 81.49847 81.49847 81.49847 81.49847 85.16820 84.55657 84.70948 84.86239
## [9] 84.70948 85.01529 85.16820 85.32110 85.16820 85.16820 85.32110 85.16820
## [17] 85.16820 85.16820 85.01529 85.01529

```

```

knn_value = c(1:20)
plot(knn_value, accuracy_vector)

```



```

max(accuracy_vector)

```

```

## [1] 85.3211

```

```

which.max(accuracy_vector)

```

```

## [1] 12

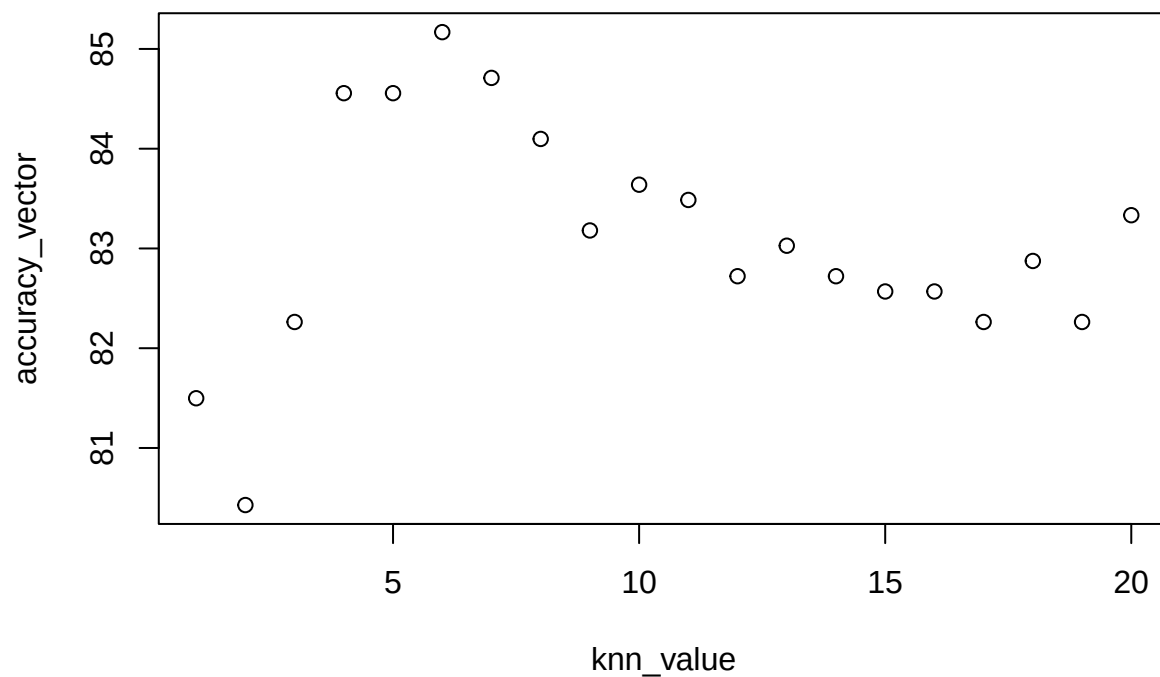
```

Rectangular Kernel

```
accuracy_knn = function(x) {  
  pred = rep(0, (nrow(data))) #Create a vector of 0's to store in the predicted  
  
  for (i in 1:nrow(data)) {  
  
    model = kkn(R1 ~. , train = data[-i,], test = data[i,], k = x, scale = TRUE, kernel = "rectangular")  
  
    pred[i] <- as.integer(fitted(model)+0.5)  
  
  }  
  
  accuracy = sum(pred == data[,11]) / nrow(data)  
  return(accuracy)  
}  
  
accuracy_vector <- 1:20  
for (x in 1:20){  
  accuracy_vector[x] = accuracy_knn(x) * 100  
}  
accuracy_vector
```

```
## [1] 81.49847 80.42813 82.26300 84.55657 84.55657 85.16820 84.70948 84.09786  
## [9] 83.18043 83.63914 83.48624 82.72171 83.02752 82.72171 82.56881 82.56881  
## [17] 82.26300 82.87462 82.26300 83.33333
```

```
knn_value = c(1:20)  
plot(knn_value, accuracy_vector)
```



```
max(accuracy_vector)
```

```
## [1] 85.1682
```

```
which.max(accuracy_vector)
```

```
## [1] 6
```